TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP THÁI NGUYÊN

KHOA ĐIỆN TỬ

BỘ MÔN: CÔNG NGHỆ THÔNG TIN

---------- 🕮 ----------

BÀI TẬP LỚN

MÔN HỌC

XỬ LÝ ẢNH

ĐỀ TÀI: BIỂU DIỄN VÀ NÉN ẢNH DỰA TRÊN BIÊN

GIÁO VIÊN HƯỚNG DẪN: ThS. ĐẶNG THỊ HIÊN

SINH VIÊN THỰC HIỆN : LƯƠNG HOÀNG VIỆT

MSSV: K225480106073

LỚP : 58.KTP

Thái Nguyên - 2026

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP THÁI NGUYÊN

KHOA ĐIỆN TỬ

BỘ MÔN: CÔNG NGHỆ THÔNG TIN

---------- 🕮 ----------

BÀI TẬP LỚN

MÔN HỌC

XỬ LÝ ẢNH

ĐỀ TÀI: BIỂU DIỄN VÀ NÉN ẢNH DỰA TRÊN BIÊN

GIÁO VIÊN HƯỚNG DẪN: ThS. ĐẶNG THỊ HIÊN

SINH VIÊN THỰC HIỆN : LƯƠNG HOÀNG VIỆT

MSSV: K225480106073

LỚP : 58.KTP

Thái Nguyên - 2026

TRƯỜNG ĐH KỸ THUẬT CÔNG NGHIỆP KHOA ĐIỆN TỬ **BỘ MÔN CÔNG NGHỆ THÔNG TIN**	**CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM** **Độc lập - Tự do - Hạnh phúc** **----o0o----**

Thái Nguyên, ngày 26 tháng 01 năm 2026

PHIẾU GIAO ĐỀ TÀI

Họ tên sinh viên: Lương Hoàng Việt MSSV: K225480106073

Chuyên ngành : KTPM Lớp học phần: 58KTP

TÊN ĐỀ TÀI: BIỂU DIỄN VÀ NÉN ẢNH DỰA TRÊN BIÊN

1. Nội dung thực hiện:

- Tìm hiểu đề tài
- Xây dựng cơ sở lý thuyết
- Xây dựng chương trình

2. Kết quả dự kiến:

- Mã nguồn chương trình.
- Bộ ảnh đầu vào và ảnh kết quả.
- File báo cáo bài tập lớn.

Ngày giao nhiệm vụ: 26/01/2026

Ngày hoàn thành nhiệm vụ: 2 4 /02/2026

Họ và tên cán bộ hướng dẫn:

BM. CÔNG NGHỆ THÔNG TIN	CÁN BỘ HƯỚNG DẪN
	Đặng Thị Hiên

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

……
……
……
……
……
……
……
……
……
……

Thái Nguyên, ngày….tháng…..năm 2026

GIÁO VIÊN HƯỚNG DẪN

(Ký ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN CHẤM

……
……
……
……
……
……
……
……
……
……

Thái Nguyên, ngày….tháng…..năm 2026

GIÁO VIÊN CHẤM

(Ký ghi rõ họ tên)

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI 3
DANH MỤC HÌNH ẢNH VÀ BẢNG BIỂU 8
LỜI NÓI ĐẦU 9
CHƯƠNG I. GIỚI THIỆU BÀI TOÁN VÀ MỤC TIÊU 10
1.1. Đặt vấn đề 10
1.2. Mục tiêu nghiên cứu 10
1.3. Nội dung thực hiện 11
1.4. Nội dung báo cáo 11
CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ THUẬT TOÁN 12
2.1. Tổng quan về biểu diễn và nén ảnh dựa trên biên 12
2.1.1. Khái niệm biên trong ảnh 12
2.1.2. Quy trình hệ thống biểu diễn và nén ảnh dựa trên biên 12
2.2. Phương pháp phát hiện biên bằng thuật toán Canny 14
2.2.1. Khái niệm 14
2.2.2. Quy trình xử lý của thuật toán Canny 14
2.2.3. Sơ đồ khối của thuật toán Canny 16
2.2.4. Ưu điểm và nhược điểm 16
2.3. Biểu diễn và nén ảnh bằng cấu trúc Mã tứ phân (Quad-tree) 17
2.3.1. Khái niệm và cấu trúc cây 17
2.3.2. Quy trình xử lý của Mã tứ phân 18
2.3.3. Sơ đồ khối của phương pháp biểu diễn ảnh bằng Mã tứ phân (Quad-Tree) 19
2.3.4. Ưu điểm và nhược điểm 19
2.4. Kỹ thuật mã hóa và đóng gói dữ liệu mức Bit (Bitwise Packing) 20
2.4.1. Đặt vấn đề 20
2.4.2. Nguyên lý hoạt động 21
2.4.3. Sơ đồ khối thuật toán Bitwise Packing 21
2.5. Các tiêu chí đánh giá chất lượng nén ảnh 23
2.5.1. Tỷ lệ nén (Compression Ratio - CR) 23
2.5.2. Sai số toàn phương trung bình (MSE) và Tỷ số tín hiệu trên nhiễu (PSNR) 23
CHƯƠNG III. CÀI ĐẶT VÀ THỰC NGHIỆM 24
3.1. Môi trường và công cụ phát triển 24
3.2. Quá trình thực nghiệm 24
3.3. Đánh giá và phân tích kết quả 27
3.4. Nhận xét và phân tích 28
CHƯƠNG IV: ĐÁNH GIÁ VÀ KẾT LUẬN 29

4.1. Kết luận 29
4.2. Hướng phát triển 30
QR CODE SẢN PHẨM CỦA BÀI TẬP LỚN 31
TÀI LIỆU THAM KHẢO 32
MÃ NGUỒN CHƯƠNG TRÌNH 33

DANH MỤC HÌNH ẢNH VÀ BẢNG BIỂU

Hình 2.1. Sơ đồ khối tổng thể của hệ thống 13
Hình 2.2. Ảnh gốc (trái) và ảnh đã được xử lý (phải) bằng Canny 14
Hình 2.3. Sơ đồ khối thuật toán phát hiện biên Canny 16
Hình 2.4. Ví dụ minh họa về quy trình xử lý của Mã tứ phân 18
Hình 2.5. Sơ đồ khối của phương pháp biểu diễn ảnh bằng Mã tứ phân 19
Hình 2.6. Sơ đồ khối của thuật toán Bitwise Packing 22
Hình 3.1. Ảnh gốc 25
Hình 3.2. Ảnh biên được xử lý bằng thuật toán Canny 25
Hình 3.3. File bin lưu thông số ảnh biên đã được nén 26
Hình 3.4. Thông số nén 26
Hình 3.5. Ảnh biên ban đầu (trái) và ảnh đã giải mã từ file bin(phải) 27
Bảng 3.1. Thống kê chỉ số thực nghiệm nén ảnh 27

LỜI NÓI ĐẦU

Trong kỷ nguyên bùng nổ của công nghệ số, lượng dữ liệu hình ảnh được tạo ra và lưu trữ đang gia tăng với tốc độ chóng mặt. Điều này đặt ra một thách thức lớn về mặt không gian bộ nhớ và băng thông truyền tải, đòi hỏi các phương pháp nén dữ liệu ngày càng tối ưu nhằm giảm thiểu tối đa dung lượng nhưng vẫn phải đảm bảo tính toàn vẹn của các thông tin quan trọng.

Thực tế cho thấy, trong nhiều ứng dụng đặc thù như hệ thống nhận dạng ký tự, xử lý tài liệu, thị giác máy tính hay các thiết bị nhúng, thông tin mang giá trị cấu trúc cốt lõi nhất của một bức ảnh thường tập trung tại các đường biên – khu vực xảy ra sự thay đổi đột ngột về cường độ sáng. Xuất phát từ thực tiễn đó, em đã quyết định lựa chọn đề tài **"Biểu diễn và nén ảnh dựa trên biên"** làm bài tập lớn của mình. Mục tiêu của đề tài là nghiên cứu sâu về các phương pháp phát hiện biên, biểu diễn cấu trúc không gian bằng thuật toán phân rã và ứng dụng chúng vào bài toán nén ảnh nhị phân, qua đó đánh giá hiệu quả thực nghiệm so với phương pháp lưu trữ truyền thống.

Để hoàn thành tốt bài tập lớn này, em xin gửi lời cảm ơn chân thành đến Ban Giám hiệu Trường Đại học Kỹ thuật Công nghiệp Thái Nguyên, cùng các thầy cô trong Khoa Điện tử và Bộ môn Công nghệ thông tin đã tạo điều kiện học tập và trang bị cho em những kiến thức nền tảng vững chắc.

Đặc biệt, em xin gửi lời tri ân sâu sắc đến cô giáo hướng dẫn – ThS. Đặng Thị Hiên. Nhờ có sự định hướng chuyên môn tận tình và những lời góp ý quý báu của cô, em mới có thể giải quyết được các vướng mắc trong thuật toán và hoàn thiện chương trình một cách trọn vẹn.

Mặc dù đã dành nhiều tâm huyết trong việc tìm hiểu lý thuyết và lập trình cài đặt hệ thống, nhưng do những hạn chế nhất định về mặt kiến thức chuyên sâu và kinh nghiệm thực tiễn, báo cáo chắc chắn vẫn còn những thiếu sót. Em rất mong nhận được sự quan tâm, đánh giá và những ý kiến đóng góp từ quý thầy cô để hệ thống cũng như kỹ năng của em ngày càng hoàn thiện hơn.

Em xin chân thành cảm ơn!

CHƯƠNG I. GIỚI THIỆU BÀI TOÁN VÀ MỤC TIÊU

1.1. Đặt vấn đề

Trong thời đại công nghệ số hiện nay, ảnh số đang được ứng dụng rộng rãi trong rất nhiều lĩnh vực đa dạng như truyền thông đa phương tiện, y tế (X-quang, CT, MRI), giám sát an ninh, viễn thám và trí tuệ nhân tạo. Kéo theo đó, lượng dữ liệu hình ảnh được tạo ra và cần lưu trữ ngày càng trở nên khổng lồ. Tuy nhiên, các bức ảnh số thường chiếm một mức dung lượng rất lớn, đặc biệt là đối với những hình ảnh có độ phân giải cao.

Chính vì vậy, bài toán nén ảnh nhằm mục đích giảm thiểu dung lượng lưu trữ cũng như tối ưu hóa quá trình truyền tải đang là một vấn đề vô cùng quan trọng trong lĩnh vực xử lý ảnh. Việc truyền tải và lưu trữ này luôn đòi hỏi các phương pháp nén thực sự hiệu quả để vừa giảm được dung lượng phần cứng, vừa phải đảm bảo duy trì được chất lượng của các thông tin quan trọng.

Thông thường, các chuẩn nén ảnh phổ biến (ví dụ như JPEG) thường hoạt động dựa trên các phép biến đổi trong miền tần số. Tuy nhiên, một hướng tiếp cận hoàn toàn khác và hiệu quả trong nhiều bài toán đặc thù là: thay vì phải lưu trữ toàn bộ thông tin điểm ảnh, hệ thống chỉ cần lưu lại thông tin quan trọng nhất, đó chính là đường biên của các đối tượng. Lý do là bởi trong rất nhiều ứng dụng thực tế như nhận dạng ký tự, xử lý tài liệu, thị giác máy tính hay các hệ thống nhúng, phần lớn thông tin cấu trúc cốt lõi của bức ảnh đều nằm ở các đường biên – nơi xảy ra sự thay đổi đột ngột về cường độ sáng.

1.2. Mục tiêu nghiên cứu

Xuất phát từ những vấn đề trên, đề tài "Biểu diễn và nén ảnh dựa trên biên" được thực hiện với mục đích nghiên cứu các phương pháp phát hiện, biểu diễn đường biên, từ đó ứng dụng trực tiếp vào bài toán nén ảnh và đánh giá hiệu quả so với các phương pháp lưu trữ truyền thống.

Cụ thể, các mục tiêu trọng tâm của đề tài bao gồm:

- Thực hiện trích xuất và phát hiện đường biên trong ảnh.
- Biểu diễn cấu trúc bức ảnh thông qua các thông tin biên thu được.
- Thực hiện nén dữ liệu ảnh bằng phương pháp mã hóa đường biên.
- Tiến hành thực nghiệm để so sánh, đánh giá dung lượng của ảnh trước và sau khi thực hiện quá trình nén.

1.3. Nội dung thực hiện

Để đạt được các mục tiêu đã đề ra, đề tài tiến hành thực hiện các nội dung công việc cụ thể như sau:

- **Tìm hiểu lý thuyết:** Nghiên cứu cơ sở lý thuyết về các phương pháp phát hiện biên trong xử lý ảnh số.
- **Cài đặt thuật toán:** Lập trình và cài đặt thuật toán phát hiện biên (cụ thể là thuật toán Canny).
- **Biểu diễn đường biên:** Ứng dụng cấu trúc dữ liệu Mã tứ phân (Quad-tree) để biểu diễn các đường biên đã được trích xuất.
- **Thực nghiệm:** Tiến hành chạy thử nghiệm chương trình trên các tập ảnh đầu vào khác nhau.
- **Đánh giá:** Tính toán các thông số và đánh giá hiệu quả nén của phương pháp đề xuất.

1.4. Nội dung báo cáo

Để trình bày một cách logic và bám sát tiến trình thực hiện, nội dung của báo cáo được tổ chức lại thành 4 chương chính như sau:

- **Chương I: Giới thiệu bài toán và mục tiêu.** Trình bày tổng quan về bối cảnh, lý do chọn đề tài, các mục tiêu cần đạt được và nội dung thực hiện cốt lõi.
- **Chương 2: Cơ sở lý thuyết và thuật toán.** Cung cấp các kiến thức nền tảng về khái niệm biên, phân tích toán tử phát hiện biên (Canny) và nguyên lý hoạt động của cấu trúc cây tứ phân (Quad-tree) trong việc phân tách không gian ảnh.
- **Chương 3: Cài đặt và thực nghiệm.** Mô tả chi tiết môi trường phát triển, lưu đồ thuật toán, quy trình xử lý nén/giải nén, đồng thời trình bày các kết quả thực nghiệm đạt được (dung lượng file, tỷ lệ nén CR, hình ảnh minh họa trước và sau giải nén).
- **Chương 4: Đánh giá và kết luận.** Tổng kết những kết quả hệ thống đã giải quyết được, đánh giá ưu/nhược điểm của phương pháp và đề xuất các hướng phát triển, mở rộng trong tương lai.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ THUẬT TOÁN

2.1. Tổng quan về biểu diễn và nén ảnh dựa trên biên

2.1.1. Khái niệm biên trong ảnh

Biên (Edge) là nơi có sự thay đổi đột ngột về mức xám trong bức ảnh. Về mặt toán học, biên được xác định là vị trí có giá trị $|\nabla f(x, y)|$ lớn. Trong biểu thức này:

- $f(x, y)$ là giá trị mức xám của điểm ảnh tại tọa độ không gian (x, y).
- ∇f là gradient biểu thị hướng và mức độ thay đổi của cường độ sáng.

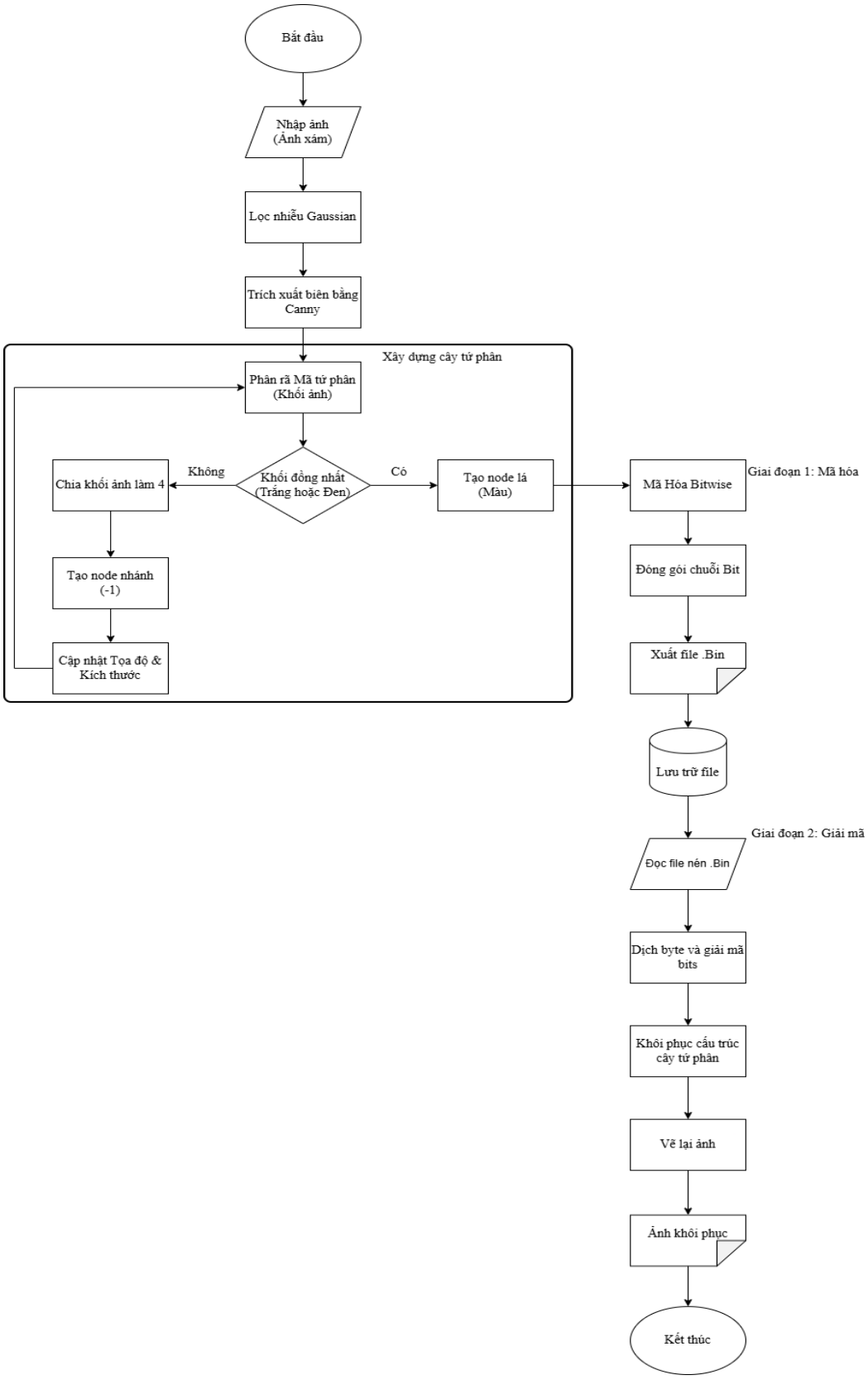
Việc xác định và giữ lại đường biên giúp lọc bỏ những thông tin dư thừa, tạo tiền đề quan trọng cho quá trình nén ảnh sau này.

2.1.2. Quy trình hệ thống biểu diễn và nén ảnh dựa trên biên

Để giải quyết bài toán giảm thiểu dung lượng lưu trữ hình ảnh mà vẫn bảo toàn được thông tin cấu trúc, hệ thống được thiết kế với một luồng xử lý khép kín bao gồm hai giai đoạn chính: Giai đoạn mã hóa (Nén ảnh) và Giai đoạn giải mã (Khôi phục ảnh). Luồng hoạt động được trình bày ngắn gọn như sau:

Nhập ảnh xám ➔ Phát hiện biên (Canny) ➔ Ảnh biên nhị phân ➔ Nén ảnh (Mã tứ phân Quad-tree) ➔ Xuất file nén (.bin) ➔ Đọc file nén ➔ Giải mã ➔ Ảnh khôi phục.

Sơ đồ khối tổng thể mô tả luồng đi của dữ liệu qua các thuật toán trong hệ thống:



Hình 2.1. Sơ đồ khối tổng thể của hệ thống

2.2. Phương pháp phát hiện biên bằng thuật toán Canny

2.2.1. Khái niệm

Khái niệm: Thuật toán Canny là phương pháp phát hiện biên đa tầng (5 bước) phổ biến nhất trong xử lý ảnh, được John F. Canny phát triển năm 1986 nhằm trích xuất các cạnh rõ nét, mỏng và chính xác. Thuật toán giúp giảm nhiễu, tính gradient, loại bỏ pixel không phải cực đại (non-maximum suppression) và sử dụng ngưỡng kép (hysteresis thresholding) để xác định cạnh thực.

Ví dụ minh họa:

Hình 2.2. Ảnh gốc (trái) và ảnh đã được xử lý (phải) bằng Canny

2.2.2. Quy trình xử lý của thuật toán Canny

Quy trình xử lý của thuật toán Canny bao gồm 5 bước tiêu chuẩn:

1. Làm mịn bằng Gaussian: Áp dụng bộ lọc không gian Gaussian để làm mờ ảnh, giúp giảm thiểu nhiễu và tránh việc thuật toán nhận diện nhầm các điểm nhiễu vụn vặt thành biên.

2. Tính gradient: Xác định cường độ và hướng của biên tại từng điểm ảnh. Độ lớn gradient G được tính toán thông qua đạo hàm theo hai hướng x và y theo công thức:

$$G = \sqrt{\left(G_x^2 + G_y^2\right)}$$

Mặt nạ theo trục x

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Mặt nạ theo trục y

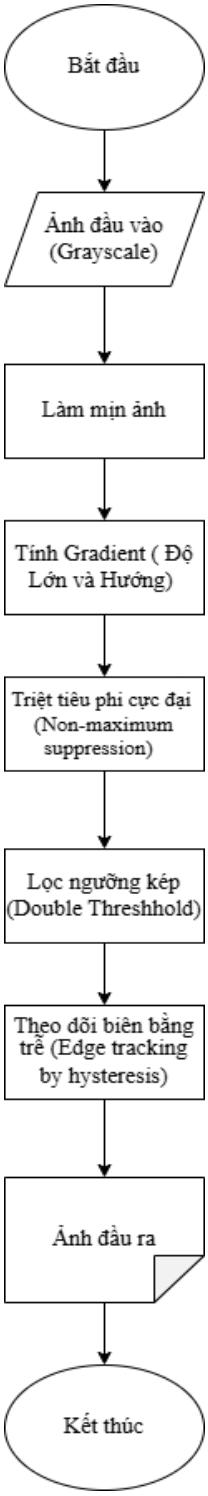
$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

3. Non-maximum suppression (Triệt tiêu phi cực đại): Thuật toán quét qua toàn bộ hình ảnh để làm mảnh các đường biên bằng cách chỉ giữ lại các điểm có giá trị cực đại cục bộ dọc theo hướng của gradient.

4. Double threshold (Ngưỡng kép): Phân loại các điểm ảnh dựa trên hai mức ngưỡng (ngưỡng cao và ngưỡng thấp). Nếu giá trị G > T (với T là ngưỡng), điểm ảnh đó được xác định chắc chắn là biên. Ngưỡng cao giúp giữ lại các đường biên chính, tạo ra các mảng đen lớn trên ảnh.

5. Edge tracking by hysteresis (Theo dõi biên bằng trễ): Liên kết các điểm biên yếu (nằm giữa hai ngưỡng) với các điểm biên mạnh để tạo thành các đường biên liền mạch, đồng thời loại bỏ các điểm nhiễu không liên kết.

2.2.3. Sơ đồ khối của thuật toán Canny



Hình 2.3. Sơ đồ khối thuật toán phát hiện biên Canny

2.2.4. Ưu điểm và nhược điểm

1. Ưu điểm: Trong đề tài này, thuật toán Canny được ưu tiên lựa chọn vì nó tối ưu và khắc phục được nhiều nhược điểm của các phương pháp khác (như toán tử Sobel). Cụ thể:

- **Cho ra đường biên mảnh:** Nhờ bước triệt tiêu phi cực đại (Non-maximum suppression), Canny chỉ giữ lại các điểm có giá trị gradient lớn nhất, giúp đường biên thu được rất sắc nét và thường chỉ rộng 1 pixel. Điều này mang ý nghĩa sống còn đối với đề tài, vì biên càng mảnh thì cấu trúc Mã tứ phân (Quad-tree) phía sau nén càng hiệu quả.
- **Ít bị ảnh hưởng bởi nhiễu:** Nhờ bước làm mịn ảnh bằng bộ lọc Gaussian ngay từ đầu quy trình, thuật toán loại bỏ được các nhiễu vụn vặt, tránh việc phát hiện sai các đường biên giả.
- **Độ chính xác cao:** Cơ chế sử dụng ngưỡng kép (Double threshold) kết hợp với theo dõi biên bằng trễ (Edge tracking by hysteresis) giúp thuật toán liên kết các điểm rời rạc thành một đường biên liền mạch, giữ lại cấu trúc thực sự của đối tượng.

2. Nhược điểm (Hạn chế): Mặc dù là một chuẩn mực trong phát hiện biên, Canny vẫn tồn tại một số hạn chế nhất định:

- **Độ phức tạp tính toán cao:** Do phải trải qua một quy trình xử lý bao gồm 5 bước phức tạp, thuật toán Canny tiêu tốn nhiều thời gian và tài nguyên CPU hơn so với các bộ lọc đơn giản như Sobel. Điều này có thể là một trở ngại khi xử lý video thời gian thực hoặc chạy trên phần cứng yếu.
- **Phụ thuộc vào việc tinh chỉnh tham số thủ công:** Kết quả của Canny phụ thuộc rất nhiều vào việc chọn giá trị ngưỡng cao (high threshold) và ngưỡng thấp (low threshold). Nếu chọn không khéo, ảnh có thể bị mất các biên quan trọng hoặc giữ lại quá nhiều rác.

2.3. Biểu diễn và nén ảnh bằng cấu trúc Mã tứ phân (Quad-tree)

Sau khi đi qua bộ lọc Canny, đầu ra thu được là một ảnh nhị phân chứa các đường biên mảnh. Trong đó, các điểm ảnh mang giá trị 255 biểu diễn đường biên (màu trắng) và các điểm ảnh mang giá trị 0 biểu diễn nền (màu đen). Để lưu trữ và truyền tải bức ảnh này một cách tối ưu, đề tài sử dụng phương pháp biểu diễn bằng Mã tứ phân (Quad-tree).

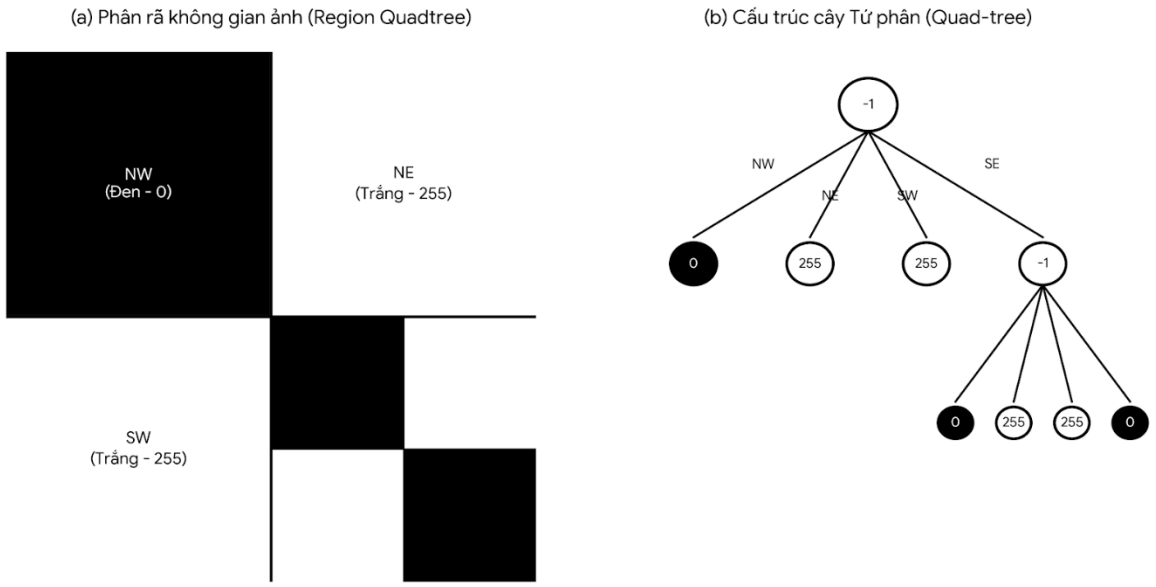
2.3.1. Khái niệm và cấu trúc cây

Khái niệm: Mã tứ phân (Quad-tree) là một cấu trúc dữ liệu dạng cây, trong đó mỗi node cha (node nhánh) sẽ có đúng 4 node con. Trong xử lý ảnh, cấu trúc này được dùng để phân chia không gian 2D của bức ảnh thành các vùng nhỏ hơn dựa trên đặc tính màu sắc của pixel.

Dựa trên cấu trúc lập trình, các node trong cây được quy định như sau:

- **Node nhánh (Mang giá trị -1):** Đại diện cho một vùng ảnh không đồng nhất (hỗn hợp), cần phải tiếp tục chia nhỏ.
- **Node lá (Mang giá trị 0 hoặc 255):** Đại diện cho một vùng ảnh hoàn toàn đồng nhất. Giá trị 0 tương ứng với một mảng nền đen hoàn toàn, và 255 tương ứng với một khối đường biên trắng.

Ví dụ minh họa:



Hình 2.4. Ví dụ minh họa về quy trình xử lý của Mã tứ phân

2.3.2. Quy trình xử lý của Mã tứ phân

Quá trình xây dựng cây tứ phân hoạt động dựa trên phương pháp đệ quy "chia để trị" với nguyên lý như sau:

Bước 1: Bắt đầu với toàn bộ bức ảnh là một vùng duy nhất (Root).

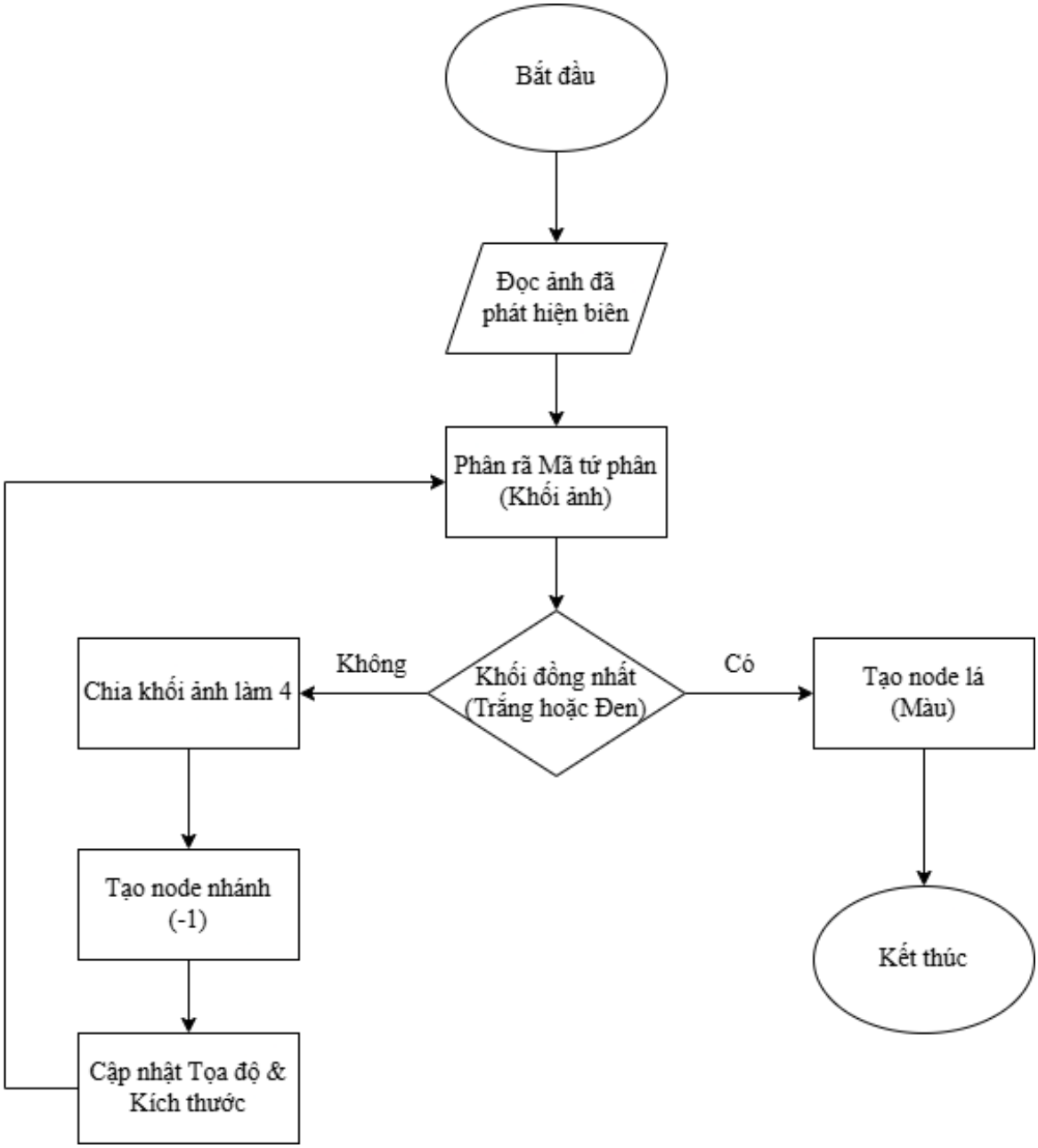
Bước 2: Kiểm tra tính đồng nhất: Thuật toán duyệt qua vùng ảnh hiện tại để kiểm tra xem tất cả các pixel trong vùng có cùng một giá trị (toàn 0 hoặc toàn 255) hay không.

Bước 3: Điều kiện dừng: Nếu vùng ảnh đó là đồng nhất, thuật toán sẽ không chia tiếp. Nó tạo ra một node lá lưu trữ giá trị màu sắc đó và kết thúc nhánh đệ quy.

Bước 4: Phân chia không gian: Nếu vùng ảnh không đồng nhất (chứa cả nền đen và đường biên trắng), thuật toán bắt buộc phải chia vùng đó thành 4 vùng

con bằng nhau theo các hướng: Tây Bắc (NW), Đông Bắc (NE), Tây Nam (SW) và Đông Nam (SE). Quá trình này tiếp tục đệ quy lặp lại từ Bước 2 cho từng vùng con.

2.3.3. Sơ đồ khối của phương pháp biểu diễn ảnh bằng Mã tứ phân (Quad-Tree)



Hình 2.5. Sơ đồ khối của phương pháp biểu diễn ảnh bằng Mã tứ phân

2.3.4. Ưu điểm và nhược điểm

1. Ưu điểm:

- **Hiệu suất nén cao ở các vùng đồng nhất:** Cấu trúc Mã tứ phân nén rất tốt các vùng đồng nhất , đặc biệt phù hợp với những bức ảnh có nhiều mảng

nền lớn. Đối với ảnh biên sau khi qua bộ lọc Canny (vốn chứa diện tích nền đen cực lớn), thuật toán Quad-tree đã gom nhóm thành công các khoảng trống rộng lớn này thành các node đơn lẻ. Điều này giúp giảm thiểu đáng kể số lượng phần tử cần lưu trữ so với phương pháp lưu trữ ma trận điểm ảnh truyền thống.

- **Bảo toàn dữ liệu tuyệt đối (Lossless Compression):** Quá trình giải mã và tái tạo ảnh từ cây tứ phân diễn ra hoàn toàn chính xác, đảm bảo tính toàn vẹn dữ liệu đối với ảnh biên nhị phân gốc. Phương pháp này không làm suy giảm hay mất mát bất kỳ điểm biên quan trọng nào của đối tượng.

2. Nhược điểm (Hạn chế):

Bên cạnh những ưu điểm vượt trội đối với ảnh biên, cấu trúc Quad-tree vẫn tồn tại một số hạn chế khi áp dụng trong thực tế:

- **Kém hiệu quả với ảnh có độ chi tiết quá dày đặc:** Nếu bức ảnh đầu vào (hoặc ảnh biên) chứa quá nhiều chi tiết đan xen, nhiễu hạt, hệ thống sẽ hiếm khi tìm được vùng đồng nhất. Khi đó, cây tứ phân buộc phải đệ quy chia nhỏ liên tục đến tận mức pixel đơn lẻ. Việc này vô tình tạo ra "tác dụng ngược", khiến dung lượng lưu trữ phình to hơn cả ảnh gốc do phải gánh thêm chi phí lưu trữ thông tin cấu trúc cây (các trỏ node nhánh).
- **Chi phí tính toán đệ quy:** Do bản chất của thuật toán là đệ quy và chia nhỏ liên tục, Quad-tree tiêu tốn nhiều tài nguyên CPU và thời gian xử lý để xây dựng/giải mã cây hơn so với việc duyệt ma trận 2D tuần tự.
- **Nhạy cảm với phép dịch chuyển (Shift variance):** Chỉ cần đối tượng trong ảnh dịch chuyển đi 1 pixel, ranh giới phân chia các khối của cây tứ phân có thể bị thay đổi hoàn toàn, làm thay đổi cấu trúc cây dù bản chất hình ảnh không đổi.

2.4. Kỹ thuật mã hóa và đóng gói dữ liệu mức Bit (Bitwise Packing)

2.4.1. Đặt vấn đề

Mặc dù cấu trúc Mã tứ phân (Quad-tree) đã biểu diễn thành công không gian ảnh thành một chuỗi các bit nhị phân (0 và 1), nhưng hệ thống tệp tin của máy tính không cho phép ghi dữ liệu ở mức độ từng bit lẻ tẻ. Đơn vị lưu trữ nhỏ nhất của đĩa cứng vật lý là 1 Byte (tương đương 8 bits).

Nếu chương trình sử dụng các kiểu dữ liệu mặc định như ký tự (char) hay giá trị logic (boolean) để lưu trữ từng trạng thái của cây, hệ điều hành sẽ tự động cấp phát

1 Byte cho mỗi trạng thái đó. Hậu quả là dung lượng file lưu trữ sẽ bị phình to gấp 8 lần so với số bit thực tế, làm mất đi hoàn toàn mục tiêu giảm dung lượng của hệ thống. Do đó, kỹ thuật đóng gói Bitwise là bắt buộc phải có để "ép" dữ liệu xuống mức cực hạn.

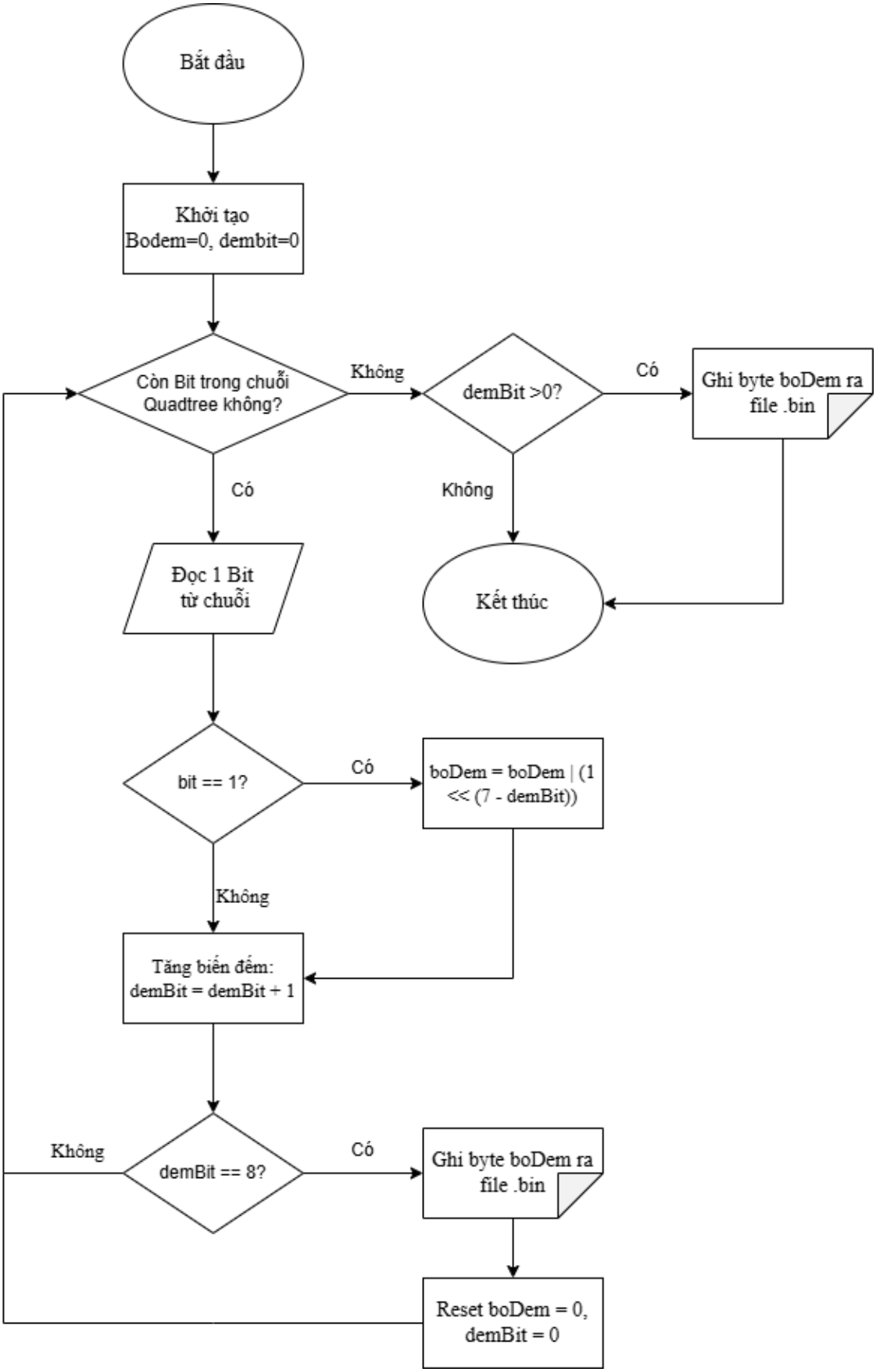
2.4.2. Nguyên lý hoạt động

Kỹ thuật Bitwise Packing can thiệp trực tiếp vào bộ nhớ RAM thông qua các phép toán thao tác bit (Bitwise operators) để nhồi chặt 8 trạng thái bit riêng lẻ vào đúng một khối 1 Byte trước khi ghi xuống ổ cứng. Quy trình này bao gồm các cơ chế sau:

- **Sử dụng biến đệm (Buffer):** Khởi tạo một biến đệm boDem có kích thước chuẩn 1 Byte (kiểu unsigned char), giá trị ban đầu là 00000000. Một biến demBit được dùng để theo dõi số lượng bit đã được nạp vào biến đệm.
- **Phép toán dịch trái (<<) và OR (|):** Thuật toán duyệt qua luồng dữ liệu của Quad-tree. Nếu bắt gặp bit mang giá trị 1, hệ thống sẽ sử dụng phép dịch trái để đẩy số 1 vào đúng ngăn tương ứng trong biến đệm, sau đó dùng phép toán OR để lưu lại trạng thái. Công thức tổng quát: boDem |= (1 << (7 - demBit))
- **Xả đệm (Flush) và Ghi file:** Khi biến đệm đã chứa đủ 8 bit (demBit == 8), hệ thống tiến hành ghi trực tiếp khối 1 Byte này vào tệp nhị phân (.bin), sau đó đặt lại boDem và demBit về 0 để tiếp tục chu trình mới.
- **Xử lý bit dư (Padding):** Tại điểm kết thúc của chuỗi dữ liệu, nếu biến đệm chưa nạp đủ 8 bit (0 < demBit < 8), thuật toán vẫn phải thực hiện lệnh ghi khối Byte đang đóng gói dở dang này xuống tệp tin để đảm bảo không có bất kỳ dữ liệu nào của cây tứ phân bị thất thoát.

2.4.3. Sơ đồ khối thuật toán Bitwise Packing

Sơ đồ dưới đây mô tả luồng xử lý chi tiết của máy trạng thái đóng gói byte trước khi xuất ra tệp nhị phân:



Hình 2.6. Sơ đồ khối của thuật toán Bitwise Packing

2.5. Các tiêu chí đánh giá chất lượng nén ảnh

Để đánh giá hiệu năng của một thuật toán nén ảnh, cần xem xét trên hai khía cạnh chính: khả năng thu gọn kích thước dữ liệu (đánh giá qua Tỷ lệ nén) và mức độ bảo toàn thông tin sau khi giải nén (đánh giá qua Lỗi toàn phương trung bình và Tỷ số tín hiệu trên nhiễu đỉnh).

2.5.1. Tỷ lệ nén (Compression Ratio - CR)

Tỷ lệ nén là đại lượng đặc trưng cho hiệu quả giảm thiểu không gian lưu trữ của thuật toán. Nó được định nghĩa là tỷ số giữa kích thước của dữ liệu gốc (chưa nén) và kích thước của dữ liệu sau khi đã được nén:

$$CR = \frac{Dung\ lượng\ ảnh\ gốc\ (Bytes)}{Dung\ lượng\ file\ nén\ (Bytes)}$$

2.5.2. Sai số toàn phương trung bình (MSE) và Tỷ số tín hiệu trên nhiễu (PSNR)

Bên cạnh việc giảm dung lượng, thuật toán nén phải đảm bảo chất lượng hình ảnh khi khôi phục. Hai đại lượng thường được sử dụng là Sai số toàn phương trung bình (MSE) và Tỷ số tín hiệu trên nhiễu đỉnh (PSNR).

MSE (Mean Squared Error) là chỉ số dùng để đo **mức sai khác trung bình bình phương giữa ảnh gốc** I **và ảnh khôi phục** K có kích thước $m \times n$:

$$MSE = \frac{1}{m*n} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i,j) - K(i,j)]^2$$

PSNR (Peak Signal-to-Noise Ratio): Được tính toán dựa trên MSE để đánh giá chất lượng ảnh tái tạo (đơn vị Decibel - dB):

$$PSNR = 10 * \log_{10}\left(\frac{MAX_I^2}{MSE}\right)$$

Trong đó, MAX_I là giá trị mức xám tối đa của điểm ảnh (đối với ảnh nhị phân 8-bit, $MAX_I = 255$).

CHƯƠNG III. CÀI ĐẶT VÀ THỰC NGHIỆM

3.1. Môi trường và công cụ phát triển

Để đảm bảo tính khách quan và hiệu năng khi xử lý ảnh mức độ điểm ảnh (pixel), hệ thống được cài đặt và thử nghiệm trên môi trường sau:

- **Ngôn ngữ lập trình:** C++
- **Môi trường phát triển (IDE):** Microsoft Visual Studio 2022.
- **Thư viện hỗ trợ:** OpenCV (Chỉ sử dụng để hỗ trợ đọc/ghi file ảnh, hiển thị cửa sổ giao diện và thao tác ma trận cơ bản. **Toàn bộ thuật toán phát hiện biên Canny và nén Quadtree đều được cài đặt thủ công**).
- **Cấu hình máy tính thử nghiệm:** RAM 16GB Intel Core I5, HĐH Windows 11.

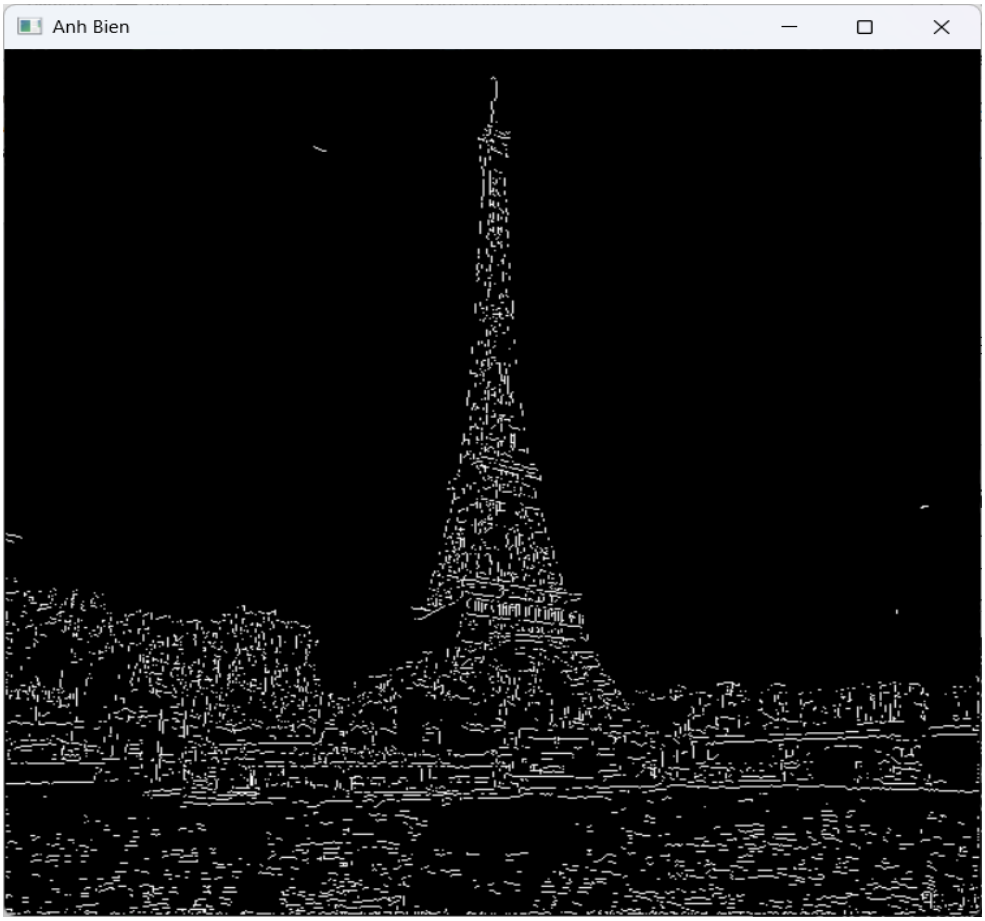
3.2. Quá trình thực nghiệm

Chương trình được chạy thử nghiệm với đầu vào là một bức ảnh đa mức xám (Grayscale). Các bước thực nghiệm được diễn ra theo đúng quy trình đã phân tích ở Chương 2.

Bước 1: Trích xuất đặc trưng (Phát hiện biên Canny): Hệ thống đọc ảnh gốc, áp dụng bộ lọc Gaussian, tính Gradient và lọc ngưỡng Hysteresis để tạo ra bức ảnh biên nhị phân (chỉ chứa giá trị pixel 0 và 255). Quá trình này được thực thi thông qua lời gọi các hàm thủ công đã cài đặt:

```
// 1. Trích xuất biên bằng bộ lọc Canny tự cài đặt
Mat anhLamMin = lamMinGaussian(anhGoc);
Mat doLon, huong;
tinhGradient(anhLamMin, doLon, huong);
Mat anhManh = trietTieuPhiCucDai(doLon, huong);
Mat anhBien = locNguongVaTheoDoi(anhManh, 60, 120);
```

Hình 3.1. Ảnh gốc



Hình 3.2. Ảnh biên được xử lý bằng thuật toán Canny

Bước 2: Nén bằng Quadtree và xuất file .bin: Hệ thống duyệt ảnh biên, xây dựng cây tứ phân và ghi luồng bit (bitstream) ra file nhị phân. Thuật toán bắt đầu

bằng việc quét toàn bộ ma trận ảnh để xây dựng cấu trúc đệ quy, sau đó dịch cây thành chuỗi Boolean:

```
// 2. Nén ảnh bằng Quadtree và mã hóa thành bitstream
NodeCayTuPhan* gocCay = xayDungCayTuPhan(anhBien, 0, 0, anhBien.cols, anhBien.rows);
vector<bool> chuoiBit;
maHoaCay(gocCay, chuoiBit);
// ... (Tiến hành dịch bit và ghi đóng gói ra file anh_da_nen.bin)
```

Sau khi ghi file, giao diện Terminal/Console hiển thị trực tiếp các thông số nén, đi kèm với file .bin được nén.

anh_da_nen.bin	12/03/2026 03:25	BIN File	21 KB

Hình 3.3. File bin lưu thông số ảnh biên đã được nén

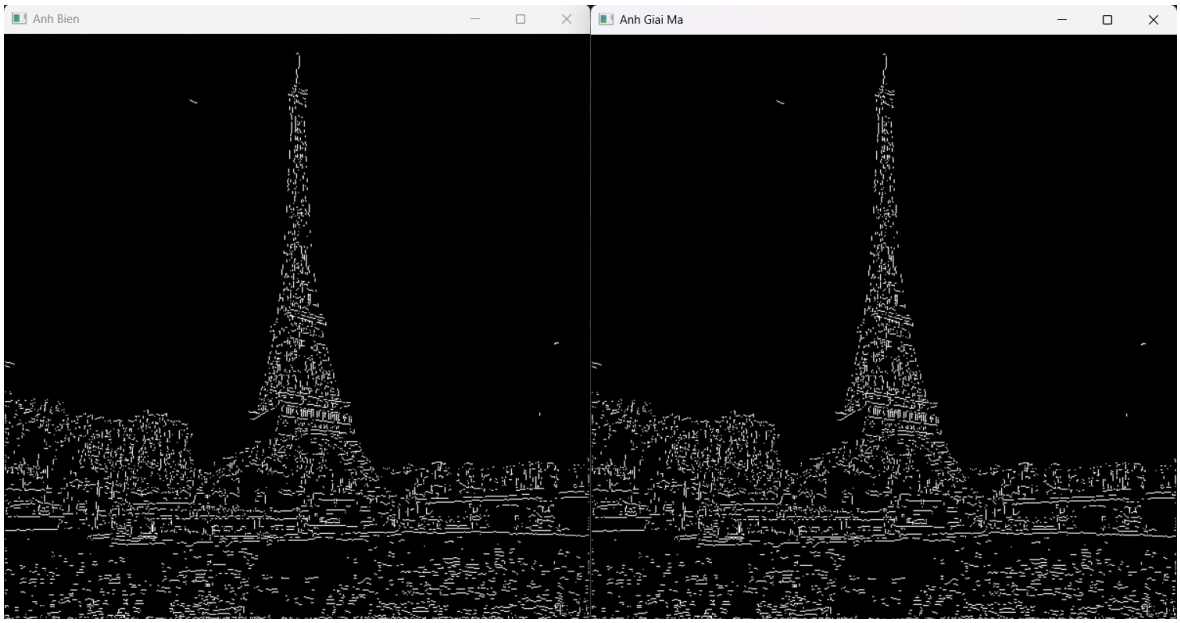
```
=================================================
            THONG KE CHI SO THUC NGHIEM
=================================================
- Kich thuoc anh bien goc    : 660000 Bytes
- Kich thuoc file .bin       : 20688 Bytes
- Ty le nen (CR)             : 31.9026 (Giam 96.8655%)
-------------------------------------------------
- Sai so toan phuong (MSE)   : 0
- Ty so tin hieu (PSNR)      : Vo cuc (Infinity) dB
  => KET LUAN: Nen Lossless (Khong mat mat) 100%
=================================================
```

Hình 3.4. Thông số nén

Bước 3: Giải mã và phục hồi ảnh: Để khôi phục, chương trình tiến hành đọc file .bin, dựng lại cây Quadtree và vẽ lại ảnh. Dữ liệu từ file nhị phân được đọc ngược lại thành chuỗi bit, từ đó giải mã để tái tạo chính xác cây đệ quy và không gian pixel:

```
// 3. Đọc bitstream từ file .bin, giải mã cây và vẽ lại ảnh
NodeCayTuPhan* gocGiaiMa = giaiMaCay(bitDocDuoc, viTri, wDoc, hDoc);
Mat anhPhucHoi = Mat::zeros(hDoc, wDoc, CV_8U);
veLaiAnhTuCay(gocGiaiMa, anhPhucHoi, 0, 0, wDoc, hDoc);
```

Ảnh sau khi phục hồi được hiển thị song song với ảnh biên ban đầu để kiểm chứng độ chính xác.



Hình 3.5. Ảnh biên ban đầu (trái) và ảnh đã giải mã từ file bin(phải)

3.3. Đánh giá và phân tích kết quả

Dựa trên các số liệu in ra từ hệ thống, ta có bảng thống kê hiệu suất nén như sau:

Chỉ số	**Giá trị**
Kích thước ảnh biên gốc	660000 Bytes
Kích thước file .bin	20688 Bytes
Tỷ lệ nén (CR)	31.9026 (Giảm 96.8655%)
Sai số toàn phương (MSE)	0
Tỷ số tín hiệu trên nhiễu (PSNR)	∞ (Infinity) dB

Bảng 3.1. Thống kê chỉ số thực nghiệm nén ảnh

3.4. Nhận xét và phân tích

Dựa vào bảng số liệu thống kê thực hiện ở mục 3.3, ta có thể rút ra những đánh giá quan trọng về hiệu năng của phương pháp biểu diễn và nén ảnh bằng cấu trúc Mã tứ phân (Quad-tree) kết hợp thuật toán phát hiện biên Canny như sau:

Về hiệu suất thu gọn kích thước (Tỷ lệ nén - CR): Hệ thống cho thấy khả năng nén cực kỳ ấn tượng với Tỷ lệ nén (CR) đạt 31.9026. Điều này đồng nghĩa với việc dung lượng lưu trữ đã được cắt giảm đến 96.8655% so với ảnh biên gốc (kích thước giảm từ 660000 Bytes xuống chỉ còn 20688 Bytes). Kết quả này đã minh chứng rõ nét cho ưu điểm lý thuyết của cấu trúc Quad-tree: sau khi đi qua toán tử Canny, ảnh biên chứa phần lớn là diện tích nền đen (mang giá trị 0). Thuật toán đã gom nhóm thành công các khoảng không gian đồng nhất rộng lớn này thành các node nhánh đơn lẻ, qua đó giảm thiểu đáng kể chi phí lưu trữ so với phương pháp lưu trữ ma trận điểm ảnh truyền thống.

Về mức độ bảo toàn thông tin khôi phục: Các chỉ số đo lường chất lượng ảnh tái tạo đều đạt mức tối ưu: Sai số toàn phương trung bình (MSE) bằng 0 và Tỷ số tín hiệu trên nhiễu đỉnh (PSNR) đạt giá trị Vô cực (Infinity). Về mặt toán học, điều này khẳng định không có bất kỳ sự sai khác hay biến dạng nào xảy ra giữa ảnh giải mã và ảnh biên gốc ban đầu. Quá trình dịch byte và khôi phục cấu trúc cây diễn ra hoàn toàn chính xác, chứng minh hệ thống đạt chuẩn Nén không mất mát (Lossless Compression) 100%.

Ảnh hưởng của đặc trưng ảnh đầu vào: Bức ảnh thực nghiệm (Tháp Eiffel) là một đối tượng có cấu trúc thép đan xen rất phức tạp, dẫn đến mật độ đường biên tại khu vực tháp là khá dày đặc. Tại các vùng này, thuật toán Quad-tree bắt buộc phải đệ quy và chia nhỏ liên tục đến tận mức pixel đơn lẻ. Do đó, phần lớn dung lượng file nén (20688 Bytes) chủ yếu bị tiêu tốn để lưu trữ cấu trúc cây tại các khu vực chi tiết này. Nếu hệ thống được áp dụng trên các dạng ảnh có mảng khối đơn giản hơn, ít nhiễu và ít chi tiết vụn (ví dụ: bản vẽ kỹ thuật mạch điện, chữ viết tài liệu), hiệu quả nén hứa hẹn sẽ còn đạt ngưỡng cao hơn nữa.

CHƯƠNG IV: ĐÁNH GIÁ VÀ KẾT LUẬN

4.1. Kết luận

Trong khuôn khổ bài tập lớn, em đã nghiên cứu và cài đặt thành công chương trình "Biểu diễn và nén ảnh dựa trên biên" sử dụng ngôn ngữ C++ và thư viện OpenCV. Hệ thống đã bám sát và giải quyết trọn vẹn các mục tiêu đặt ra từ đầu đề tài

- **Trích xuất đặc trưng hình học:** Cài đặt thủ công và áp dụng thành công thuật toán Canny để khử nhiễu, tính toán gradient và trích xuất ra các đường biên sắc nét, liền mảnh của đối tượng.
- **Hiệu suất nén vượt trội:** Áp dụng hiệu quả cấu trúc dữ liệu Mã tứ phân (Quad-tree) để biểu diễn không gian ảnh biên. Nhờ đặc thù ảnh sau khi qua toán tử Canny chứa phần lớn là diện tích nền đen, thuật toán Quad-tree đã gom nhóm tối ưu các không gian đồng nhất rộng lớn này thành các node nhánh đơn lẻ. Kết quả thực nghiệm cho thấy dung lượng file lưu trữ đã giảm mạnh từ 660.000 Bytes xuống chỉ còn 20.688 Bytes. Tỷ lệ nén (CR) đạt mức ấn tượng 31.9026, tương đương với việc tiết kiệm đến 96.86% không gian nhớ so với phương pháp ma trận điểm ảnh truyền thống.
- **Bảo toàn dữ liệu tuyệt đối (Lossless Compression):** Quá trình đóng gói bitwise, giải mã và tái tạo ảnh từ cây tứ phân diễn ra chính xác hoàn toàn. Hệ thống ghi nhận mức sai số toàn phương trung bình $MSE = 0$ và tỷ số tín hiệu trên nhiễu đỉnh $PSNR = \infty$, khẳng định thuật toán không làm mất mát hay sai lệch bất kỳ điểm ảnh nào so với ảnh biên nhị phân gốc.
- **Tiềm năng ứng dụng:** Từ các kết quả trên, có thể thấy phương pháp mã hóa này đặc biệt phù hợp và có tiềm năng ứng dụng rất lớn trong việc lưu trữ, truyền tải các bản vẽ kỹ thuật mạch điện, văn bản tài liệu dạng scan, hoặc cấu trúc ảnh y tế – những dữ liệu mà đường biên đóng vai trò chứa thông tin cốt lõi và có các mảng nền đồng nhất lớn.

Thông qua quá trình thực hiện đề tài này, em không chỉ củng cố vững chắc các nền tảng lý thuyết cốt lõi về xử lý ảnh số, rèn luyện tư duy phân tích thuật toán đệ quy dạng cây, mà còn nâng cao đáng kể kỹ năng lập trình thao tác bit và quản lý bộ nhớ trực tiếp bằng ngôn ngữ C++.

4.2. Hướng phát triển

Mặc dù chương trình đã đạt được những kết quả khả quan về mặt thuật toán cốt lõi, hệ thống vẫn có thể tiếp tục được mở rộng và tối ưu hóa theo các hướng sau trong tương lai:

- **So sánh thực nghiệm đa phương pháp:** Cài đặt thêm các thuật toán nén ảnh nhị phân khác như Mã xích (Freeman Chain Code) hoặc Mã loạt dài (Run-Length Encoding - RLE) để có cơ sở đánh giá, so sánh ưu nhược điểm và thời gian chạy thực tế giữa các phương pháp.
- **Tự động hóa ngưỡng Canny (Adaptive Thresholding):** Áp dụng các thuật toán tự động tính toán ngưỡng (dựa trên phương sai Otsu hoặc giá trị trung vị của ảnh) để tối ưu hóa việc tách biên cho nhiều loại ảnh có điều kiện ánh sáng khác nhau mà không cần người dùng tinh chỉnh thủ công.
- **Triển khai trên hệ thống nhúng:** Nhờ đặc tính quản lý phân vùng logic, thuật toán nén Quadtree có thể được tối ưu hóa để chạy trên các vi điều khiển hoặc phần cứng FPGA phục vụ cho các hệ thống giám sát IoT có băng thông truyền tải thấp.
- **Xây dựng giao diện người dùng (GUI):** Thay vì thao tác trên giao diện Console màn hình đen, hướng tới việc tích hợp các thư viện GUI (như Qt, ImGui hoặc C# WinForms) để tạo phần mềm hoàn chỉnh. Qua đó, người dùng có thể tải ảnh lên bằng nút bấm và dùng thanh trượt (slider) để tinh chỉnh ngưỡng Canny trực tiếp, quan sát kết quả nén theo thời gian thực.

QR CODE SẢN PHẨM CỦA BÀI TẬP LỚN

TÀI LIỆU THAM KHẢO

1. Rafael C. Gonzalez & Richard E. Woods – Digital Image Processing

2. John F. Canny (1986), A Computational Approach to Edge Detection

3. OpenCV Documentation

MÃ NGUỒN CHƯƠNG TRÌNH

```
#include <iostream>
#include <vector>
#include <fstream>
#include <opencv2/opencv.hpp>
#include <opencv2/core/utils/logger.hpp>

using namespace std;
using namespace cv;

// ============================================================
// PHẦN 1: CÀI ĐẶT THỦ CÔNG THUẬT TOÁN CANNY
// ============================================================

Mat lamMinGaussian(const Mat& anhGoc) {
    Mat anhKq = anhGoc.clone();
    double kernel[5][5] = { {2,4,5,4,2}, {4,9,12,9,4}, {5,12,15,12,5}, {4,9,12,9,4}, {2,4,5,4,2} };
    double tongSo = 159.0;
    for (int i = 2; i < anhGoc.rows - 2; i++) {
        for (int j = 2; j < anhGoc.cols - 2; j++) {
            double giaTri = 0;
            for (int ki = -2; ki <= 2; ki++) {
                for (int kj = -2; kj <= 2; kj++) {
                    giaTri += anhGoc.at<uchar>(i + ki, j + kj) * kernel[ki + 2][kj + 2];
                }
            }
            anhKq.at<uchar>(i, j) = (uchar)(giaTri / tongSo);
        }
    }
    return anhKq;
}

void tinhGradient(const Mat& anhLamMin, Mat& doLon, Mat& huong) {
    doLon = Mat::zeros(anhLamMin.size(), CV_32F);
    huong = Mat::zeros(anhLamMin.size(), CV_32F);
    int Gx[3][3] = { {-1,0,1}, {-2,0,2}, {-1,0,1} };
    int Gy[3][3] = { {-1,-2,-1}, {0,0,0}, {1,2,1} };
    for (int i = 1; i < anhLamMin.rows - 1; i++) {
        for (int j = 1; j < anhLamMin.cols - 1; j++) {
            float tongX = 0, tongY = 0;
            for (int ki = -1; ki <= 1; ki++) {
                for (int kj = -1; kj <= 1; kj++) {
                    int pixel = anhLamMin.at<uchar>(i + ki, j + kj);
                    tongX += pixel * Gx[ki + 1][kj + 1];
                    tongY += pixel * Gy[ki + 1][kj + 1];
                }
            }
            doLon.at<float>(i, j) = sqrt(tongX * tongX + tongY * tongY);
            huong.at<float>(i, j) = atan2(tongY, tongX) * 180.0 / CV_PI;
        }
    }
}

Mat trietTieuPhiCucDai(const Mat& doLon, const Mat& huong) {
    Mat kq = Mat::zeros(doLon.size(), CV_8U);
    for (int i = 1; i < doLon.rows - 1; i++) {
        for (int j = 1; j < doLon.cols - 1; j++) {
            float goc = huong.at<float>(i, j);
            if (goc < 0) goc += 180;
```

```
            float q = 255, r = 255;
            if ((goc >= 0 && goc < 22.5) || (goc >= 157.5 && goc <= 180)) { q = doLon.at<float>(i, j + 1); r =
doLon.at<float>(i, j - 1); }
            else if (goc >= 22.5 && goc < 67.5) { q = doLon.at<float>(i + 1, j - 1); r = doLon.at<float>(i - 1, j
+ 1); }
            else if (goc >= 67.5 && goc < 112.5) { q = doLon.at<float>(i + 1, j); r = doLon.at<float>(i - 1, j);
}
            else if (goc >= 112.5 && goc < 157.5) { q = doLon.at<float>(i - 1, j - 1); r = doLon.at<float>(i +
1, j + 1); }

            if (doLon.at<float>(i, j) >= q && doLon.at<float>(i, j) >= r) kq.at<uchar>(i, j) =
(uchar)doLon.at<float>(i, j);
            else kq.at<uchar>(i, j) = 0;
        }
    }
    return kq;
}

Mat locNguongVaTheoDoi(const Mat& anhManh, int nguongThap, int nguongCao) {
    Mat kq = Mat::zeros(anhManh.size(), CV_8U);
    uchar YEU = 50; uchar MANH = 255;
    for (int i = 0; i < anhManh.rows; i++) {
        for (int j = 0; j < anhManh.cols; j++) {
            int giaTri = anhManh.at<uchar>(i, j);
            if (giaTri >= nguongCao) kq.at<uchar>(i, j) = MANH;
            else if (giaTri >= nguongThap) kq.at<uchar>(i, j) = YEU;
        }
    }
    for (int i = 1; i < kq.rows - 1; i++) {
        for (int j = 1; j < kq.cols - 1; j++) {
            if (kq.at<uchar>(i, j) == YEU) {
                if (kq.at<uchar>(i + 1, j - 1) == MANH || kq.at<uchar>(i + 1, j) == MANH || kq.at<uchar>(i + 1,
j + 1) == MANH ||
                    kq.at<uchar>(i, j - 1) == MANH || kq.at<uchar>(i, j + 1) == MANH || kq.at<uchar>(i - 1, j -
1) == MANH ||
                    kq.at<uchar>(i - 1, j) == MANH || kq.at<uchar>(i - 1, j + 1) == MANH) {
                    kq.at<uchar>(i, j) = MANH;
                }
                else kq.at<uchar>(i, j) = 0;
            }
        }
    }
    for (int i = 0; i < kq.rows; i++) {
        for (int j = 0; j < kq.cols; j++) {
            if (kq.at<uchar>(i, j) == YEU) kq.at<uchar>(i, j) = 0;
        }
    }
    return kq;
}

// ============================================================
// PHẦN 2: CẤU TRÚC QUADTREE (MÃ TỨ PHÂN)
// ============================================================

struct NodeCayTuPhan {
    int giaTri;
    NodeCayTuPhan* con[4];
    NodeCayTuPhan(int val) {
        giaTri = val;
        for (int i = 0; i < 4; i++) con[i] = nullptr;
```

```
    }
};

int kiemTraDongNhat(const Mat& anh, int x, int y, int rong, int cao) {
    if (rong <= 0 || cao <= 0) return -1;
    Mat vung = anh(Rect(x, y, rong, cao));
    int diemSang = countNonZero(vung);
    if (diemSang == 0) return 0;
    if (diemSang == rong * cao) return 255;
    return -1;
}

NodeCayTuPhan* xayDungCayTuPhan(const Mat& anh, int x, int y, int rong, int cao) {
    if (rong <= 0 || cao <= 0) return nullptr;
    int loaiNode = kiemTraDongNhat(anh, x, y, rong, cao);

    if (loaiNode != -1 || (rong <= 1 && cao <= 1)) {
        if (loaiNode == -1) loaiNode = anh.at<uchar>(y, x);
        return new NodeCayTuPhan(loaiNode);
    }

    NodeCayTuPhan* node = new NodeCayTuPhan(-1);
    int nuaRong = rong / 2, nuaCao = cao / 2;
    int rong2 = rong - nuaRong, cao2 = cao - nuaCao;

    node->con[0] = xayDungCayTuPhan(anh, x, y, nuaRong, nuaCao);
    node->con[1] = xayDungCayTuPhan(anh, x + nuaRong, y, rong2, nuaCao);
    node->con[2] = xayDungCayTuPhan(anh, x, y + nuaCao, nuaRong, cao2);
    node->con[3] = xayDungCayTuPhan(anh, x + nuaRong, y + nuaCao, rong2, cao2);
    return node;
}

void maHoaCay(NodeCayTuPhan* node, vector<bool>& chuoiBit) {
    if (!node) return;
    if (node->giaTri == -1) {
        chuoiBit.push_back(true);
        for (int i = 0; i < 4; i++) maHoaCay(node->con[i], chuoiBit);
    }
    else {
        chuoiBit.push_back(false);
        chuoiBit.push_back(node->giaTri == 255);
    }
}

// ============================================================
// PHẦN 3: GIẢI MÃ TỪ BITSTREAM
// ============================================================

NodeCayTuPhan* giaiMaCay(const vector<bool>& chuoiBit, int& viTri, int rong, int cao) {
    if (rong <= 0 || cao <= 0) return nullptr;
    if (viTri >= chuoiBit.size()) return nullptr;

    bool laNhanh = chuoiBit[viTri++];

    if (rong <= 1 && cao <= 1) {
        if (laNhanh) {
            if (viTri < chuoiBit.size()) viTri++;
            return new NodeCayTuPhan(0);
        }
        else {
```

```
            if (viTri >= chuoiBit.size()) return new NodeCayTuPhan(0);
            bool laTrang = chuoiBit[viTri++];
            return new NodeCayTuPhan(laTrang ? 255 : 0);
        }
    }

    if (laNhanh) {
        NodeCayTuPhan* node = new NodeCayTuPhan(-1);
        int nuaRong = rong / 2, nuaCao = cao / 2;
        int rong2 = rong - nuaRong, cao2 = cao - nuaCao;

        node->con[0] = giaiMaCay(chuoiBit, viTri, nuaRong, nuaCao);
        node->con[1] = giaiMaCay(chuoiBit, viTri, rong2, nuaCao);
        node->con[2] = giaiMaCay(chuoiBit, viTri, nuaRong, cao2);
        node->con[3] = giaiMaCay(chuoiBit, viTri, rong2, cao2);
        return node;
    }
    else {
        if (viTri >= chuoiBit.size()) return new NodeCayTuPhan(0);
        bool laTrang = chuoiBit[viTri++];
        return new NodeCayTuPhan(laTrang ? 255 : 0);
    }
}

void veLaiAnhTuCay(NodeCayTuPhan* node, Mat& anh, int x, int y, int rong, int cao) {
    if (!node || rong <= 0 || cao <= 0) return;
    if (node->giaTri != -1) {
        anh(Rect(x, y, rong, cao)).setTo(node->giaTri);
        return;
    }
    int nuaRong = rong / 2, nuaCao = cao / 2;
    int rong2 = rong - nuaRong, cao2 = cao - nuaCao;
    veLaiAnhTuCay(node->con[0], anh, x, y, nuaRong, nuaCao);
    veLaiAnhTuCay(node->con[1], anh, x + nuaRong, y, rong2, nuaCao);
    veLaiAnhTuCay(node->con[2], anh, x, y + nuaCao, nuaRong, cao2);
    veLaiAnhTuCay(node->con[3], anh, x + nuaRong, y + nuaCao, rong2, cao2);
}

void giaiPhongCay(NodeCayTuPhan* node) {
    if (!node) return;
    for (int i = 0; i < 4; i++) giaiPhongCay(node->con[i]);
    delete node;
}

// ============================================================
// HÀM MAIN
// ============================================================

int main() {
    cv::utils::logging::setLogLevel(cv::utils::logging::LOG_LEVEL_ERROR);

    // --- CẤU HÌNH ĐƯỜNG DẪN TỆP TIN ---
    string path_anhGoc = "D://Xu_ly_anh//baitaplon//anhgoc.jpg";
    string path_anhBien = "D://Xu_ly_anh//baitaplon//anh_bien.jpg";
    string path_fileNen = "D://Xu_ly_anh//baitaplon//anh_da_nen.bin";
    string path_anhGiaiMa = "D://Xu_ly_anh//baitaplon//anh_giai_ma.jpg";

    cout << "--- BAT DAU CHUONG TRINH ---" << endl;

    Mat anhGoc = imread(path_anhGoc, IMREAD_GRAYSCALE);
```

```
if (anhGoc.empty()) {
    cout << "[Loi] Khong tim thay anh goc! Kiem tra lai duong dan." << endl;
    cin.get(); return -1;
}

// 1. TRÍCH XUẤT BIÊN BẰNG CANNY
cout << "1. Dang trich xuat bien bang thuat toan Canny..." << endl;
Mat anhLamMin = lamMinGaussian(anhGoc);
Mat doLon, huong;
tinhGradient(anhLamMin, doLon, huong);
Mat anhManh = trietTieuPhiCucDai(doLon, huong);
Mat anhBien = locNguongVaTheoDoi(anhManh, 60, 120);
imwrite(path_anhBien, anhBien);

// 2. MÃ HÓA QUADTREE VÀ GHI RA FILE BIN
cout << "2. Dang nen anh bang Quadtree vao file .bin..." << endl;
NodeCayTuPhan* gocCay = xayDungCayTuPhan(anhBien, 0, 0, anhBien.cols, anhBien.rows);
vector<bool> chuoiBit;
maHoaCay(gocCay, chuoiBit);

ofstream file(path_fileNen, ios::binary);
int w = anhBien.cols, h = anhBien.rows;
file.write((char*)&w, sizeof(w));
file.write((char*)&h, sizeof(h));

unsigned char boDem = 0;
int demBit = 0;
for (bool bit : chuoiBit) {
    if (bit) boDem |= (1 << (7 - demBit));
    demBit++;
    if (demBit == 8) { file.write((char*)&boDem, 1); boDem = 0; demBit = 0; }
}
if (demBit > 0) file.write((char*)&boDem, 1);

long sizeFileNen = file.tellp();
file.close();

// 3. ĐỌC VÀ GIẢI MÃ TỪ FILE BIN
cout << "3. Dang doc va giai ma tu file .bin..." << endl;
ifstream fileDoc(path_fileNen, ios::binary);

int wDoc, hDoc;
fileDoc.read((char*)&wDoc, sizeof(wDoc));
fileDoc.read((char*)&hDoc, sizeof(hDoc));
vector<bool> bitDocDuoc;

unsigned char byteDoc;
while (fileDoc.read((char*)&byteDoc, 1)) {
    for (int i = 7; i >= 0; i--) bitDocDuoc.push_back((byteDoc >> i) & 1);
}
fileDoc.close();

int viTri = 0;
NodeCayTuPhan* gocGiaiMa = giaiMaCay(bitDocDuoc, viTri, wDoc, hDoc);

Mat anhPhucHoi = Mat::zeros(hDoc, wDoc, CV_8U);
veLaiAnhTuCay(gocGiaiMa, anhPhucHoi, 0, 0, wDoc, hDoc);
imwrite(path_anhGiaiMa, anhPhucHoi);

// ============================================================
```

```
    // TÍNH TOÁN CÁC CHỈ SỐ THỐNG KÊ
    // ============================================================
    double sizeAnhGoc = w * h;
    double cr = sizeAnhGoc / (double)sizeFileNen;

    double mse = 0.0;
    for (int i = 0; i < hDoc; i++) {
        for (int j = 0; j < wDoc; j++) {
            double diff = (double)anhBien.at<uchar>(i, j) - (double)anhPhucHoi.at<uchar>(i, j);
            mse += diff * diff;
        }
    }
    mse = mse / (double)(wDoc * hDoc);

    double psnr = 0.0;
    if (mse == 0) psnr = INFINITY;
    else psnr = 10.0 * log10((255.0 * 255.0) / mse);

    cout << "\n=====================================================" << endl;
    cout << "            THONG KE CHI SO THUC NGHIEM            " << endl;
    cout << "=====================================================" << endl;
    cout << "- Kich thuoc anh bien goc   : " << sizeAnhGoc << " Bytes" << endl;
    cout << "- Kich thuoc file .bin      : " << sizeFileNen << " Bytes" << endl;
    cout << "- Ty le nen (CR)            : " << cr << " (Giam " << (1.0 - 1.0 / cr) * 100 << "%)" << endl;
    cout << "-----------------------------------------------" << endl;
    cout << "- Sai so toan phuong (MSE)  : " << mse << endl;
    if (mse == 0) {
        cout << "- Ty so tin hieu (PSNR)     : Vo cuc (Infinity) dB" << endl;
        cout << "  => KET LUAN: Nen Lossless (Khong mat mat) 100%" << endl;
    }
    else cout << "- Ty so tin hieu (PSNR)     : " << psnr << " dB" << endl;
    cout << "=====================================================\n" << endl;

    giaiPhongCay(gocCay);
    giaiPhongCay(gocGiaiMa);

    cout << "=> XU LY HOAN TAT! Hien thi anh..." << endl;
    namedWindow("Anh Bien", WINDOW_NORMAL); resizeWindow("Anh Bien", 600, 600);
    namedWindow("Anh Giai Ma", WINDOW_NORMAL); resizeWindow("Anh Giai Ma", 600, 600);
    imshow("Anh Bien", anhBien);
    imshow("Anh Giai Ma", anhPhucHoi);
    waitKey(0);

    return 0;
}
```